

Session 11 : Post-Processing et Render Targets

💡 Objectifs de la session

On a déjà croisé les render targets deux fois : en **Session 8** (miroir, eau) et en **Session 10** (la shadow map est un render target de profondeur). Aujourd’hui on les utilise pour le **post-processing** — appliquer des effets sur l’image finale **après** le rendu de la scène. C’est ce qui donne le “look” d’un jeu : tone mapping, bloom, flou de profondeur, vignette, corrections colorimétriques. On va voir le pipeline, les effets principaux, et l’implémentation en Three.js.

💡 Rappel : ce qu’on sait déjà

- **Session 8** : un render target est une texture dans laquelle le GPU rend au lieu d’afficher. Pipeline en deux passes : rendre dans la texture, puis l’utiliser.
- **Session 10** : la shadow map est un render target de profondeur — on rend la scène depuis la lumière, on ne garde que le *z*.
- **Session 5** : le fragment shader calcule la couleur finale de chaque pixel. Mais cette couleur est écrite dans le framebuffer — et si on écrivait dans une texture d’abord ?

Partie 1 : Le Pipeline de Post-Processing

1. Pourquoi rendre dans une texture ?

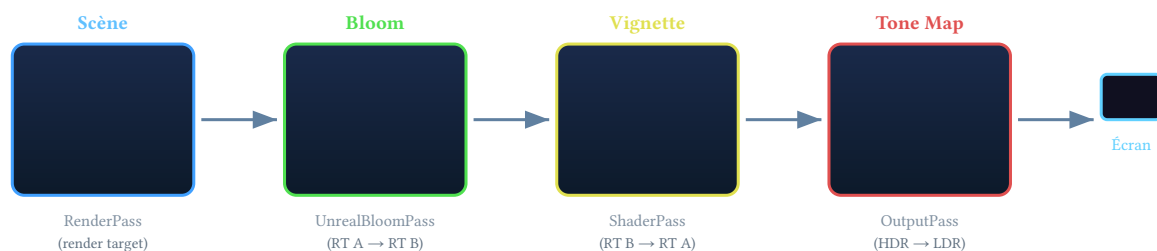
💬 Le problème

Le fragment shader produit une couleur par pixel et l’écrit directement à l’écran. Une fois affichée, cette couleur est **perdue** — on ne peut plus la modifier. Si on veut appliquer un effet (flou, bloom, tone mapping), il faut d’abord capturer l’image dans une texture, puis la traiter dans un second passage.

Le pipeline standard

1. **Passe 1 — Rendu de la scène** : la scène est rendue normalement, mais au lieu d’afficher à l’écran, on écrit dans un **render target** (couleur + profondeur).
2. **Passe 2+ — Effets** : un shader plein écran (un simple quad qui couvre tout l’écran) lit le render target, applique un effet, et écrit le résultat dans un **autre** render target.
3. **Passe finale** : le dernier render target est affiché à l’écran.

On enchaîne les effets comme une chaîne de montage : chaque effet lit l’image précédente, la transforme, produit l’image suivante.



Ping-pong : les effets alternent entre deux render targets

Figure 1: Pipeline de post-processing. La scène est rendue dans un render target, puis chaque effet lit l'image précédente et écrit dans la suivante. Le dernier résultat est affiché à l'écran.

2. Le fullscreen quad

Comment appliquer un shader sur toute l'image ?

On rend un simple rectangle (quad) qui couvre tout l'écran. Son vertex shader est trivial — il place les 4 coins aux bords de l'écran. Le fragment shader de ce quad est l'effet : il lit la texture d'entrée (l'image précédente) à la coordonnée UV du pixel courant, la transforme, et écrit le résultat.

C'est juste un mesh normal avec un matériau qui a la texture d'entrée en `sampler2D`. La géométrie n'a pas d'importance — seul le fragment shader compte.

Vertex shader du fullscreen quad.

```

// Deux triangles qui couvrent l'écran, UV en [0,1]
in vec2 aPosition; // [-1,1] en NDC
out vec2 vUV;

void main() {
    vUV = aPosition * 0.5 + 0.5; // [-1,1] → [0,1]
    gl_Position = vec4(aPosition, 0.0, 1.0);
}
  
```

3. Ping-pong rendering

Deux render targets, on alterne

On utilise deux render targets (A et B). L'effet 1 lit A, écrit dans B. L'effet 2 lit B, écrit dans A. On "ping-pong" entre les deux — pas besoin de créer une texture par effet.

$$RT_A \xrightarrow{\text{effet 1}} RT_B \xrightarrow{\text{effet 2}} RT_A \xrightarrow{\text{effet 3}} RT_B \xrightarrow{\text{afficher}} \text{écran}$$

Partie 2 : Les Effets Principaux

1. Tone Mapping — HDR vers LDR

Le problème des couleurs > 1

En rendu HDR (High Dynamic Range), les couleurs peuvent dépasser 1.0 — une lumière brillante peut produire une valeur de 5.0 ou 10.0. Mais l'écran n'affiche que [0, 1]. Sans conversion, tout ce qui dépasse 1.0 est **écrêté** (clamped à 1.0) → les hautes lumières apparaissent comme des taches blanches plates, sans détail.

Le **tone mapping** compresse les valeurs HDR vers [0, 1] tout en préservant les détails dans les hautes et basses lumières.

Les deux opérateurs courants

Reinhard — le plus simple :

$$\text{color}_{\text{LDR}} = \text{color}_{\text{HDR}} / (\text{color}_{\text{HDR}} + 1)$$

Écrase doucement les hautes lumières. Rapide, mais peut donner un rendu “plat” — il compresse trop au milieu.

ACES (Academy Color Encoding System) — le standard du cinéma :

$$\text{color}_{\text{LDR}} = \text{ACES}(\text{color}_{\text{HDR}})$$

Préserve plus de contraste et de saturation dans les hautes lumières. C'est le tone mapper par défaut dans la plupart des moteurs modernes (Unreal, Unity, Godot, Three.js).

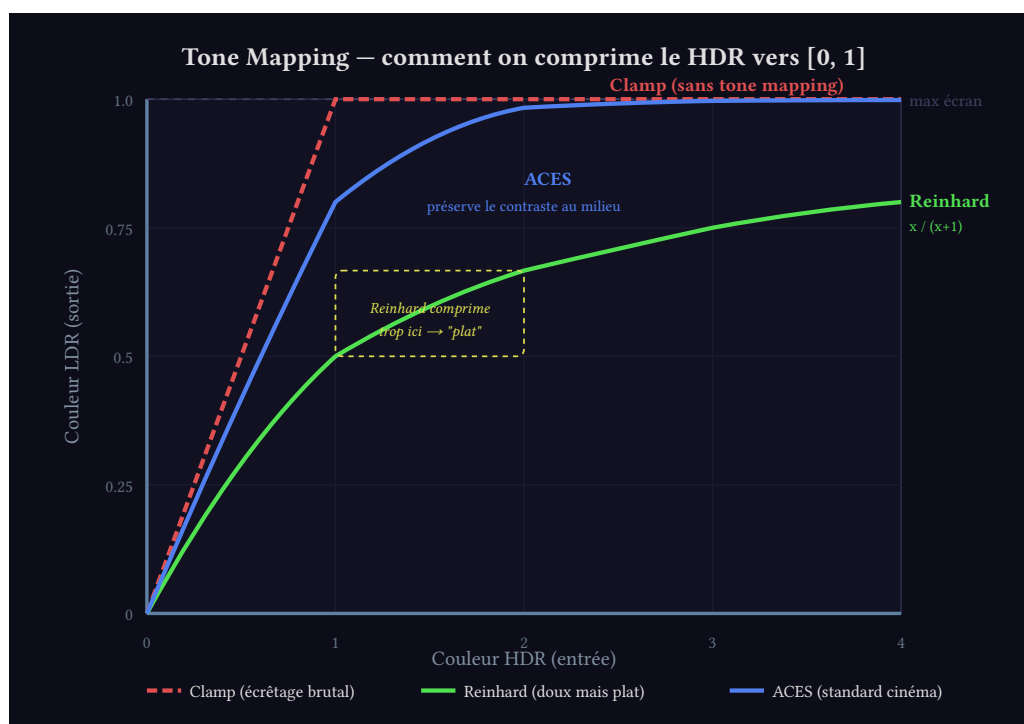


Figure 2: Courbes de tone mapping. Le **clamp** (rouge) écrête brutalement à 1.0 — les détails des hautes lumières sont perdus. **Reinhard** (vert) compresse doucement mais trop au milieu → rendu “plat”. **ACES** (bleu) reste linéaire plus longtemps, puis compresse en douceur — préserve le contraste.

Tone mapping en GLSL.

```
// Reinhard
vec3 reinhard(vec3 hdr) {
    return hdr / (hdr + 1.0);
}

// ACES (version simplifiée)
vec3 aces(vec3 x) {
    float a = 2.51, b = 0.03, c = 2.43, d = 0.59, e = 0.14;
    return clamp((x * (a * x + b)) / (x * (c * x + d) + e), 0.0, 1.0);
}
```

💡 Quand fait-on le tone mapping ?

Toujours en **dernier** — après tous les autres effets. Le tone mapping convertit l’HDR en LDR ; une fois fait, on est en $[0,1]$ et les effets suivants n’auraient plus de plage dynamique pour travailler.

2. Gamma Correction — linéaire vs sRGB

Le piège classique

Les écrans affichent en sRGB (gamma ≈ 2.2). Les calculs d’éclairage doivent se faire en **linéaire**. Si on oublie la conversion, les dégradés sont trop sombres au milieu et les lumières trop saturées.

$$\text{linéaire} \xrightarrow{1/2.2} \text{sRGB} \quad (\text{affichage})$$

$$\text{sRGB} \xrightarrow{2.2} \text{linéaire} \quad (\text{calcul})$$

💬 En pratique

En Three.js, si le `renderer.outputColorSpace` est `SRGBColorSpace` (le défaut), la conversion gamma est faite automatiquement à la fin. En Babylon.js, c’est `BABYLON.ColorSpace.SRGB`. Il faut juste s’assurer que les textures diffusent sont marquées comme sRGB et que les calculs se font en linéaire.

3. Bloom — l’halo lumineux

💬 L’effet le plus reconnaissable

Le bloom fait “rayonner” les zones lumineuses — une fenêtre au soleil, une lampe, une explosion. Sans bloom, les lumières brillantes sont des pixels blancs plats. Avec bloom, elles débordent en halo doux autour de la source.

Le pipeline du bloom

1. **Bright pass** : extraire les pixels dont la luminance dépasse un seuil (ex. > 1.0). On obtient une image ne contenant que les parties brillantes.
2. **Blur** : flouter cette image (gaussian blur, plusieurs passes pour un flou large).
3. **Composite** : additionner l’image floutée à l’image originale.

$$\text{image}_{\text{finale}} = \text{image}_{\text{scène}} + \text{image}_{\text{bright}} \xrightarrow{\text{blur}} \text{halo}$$

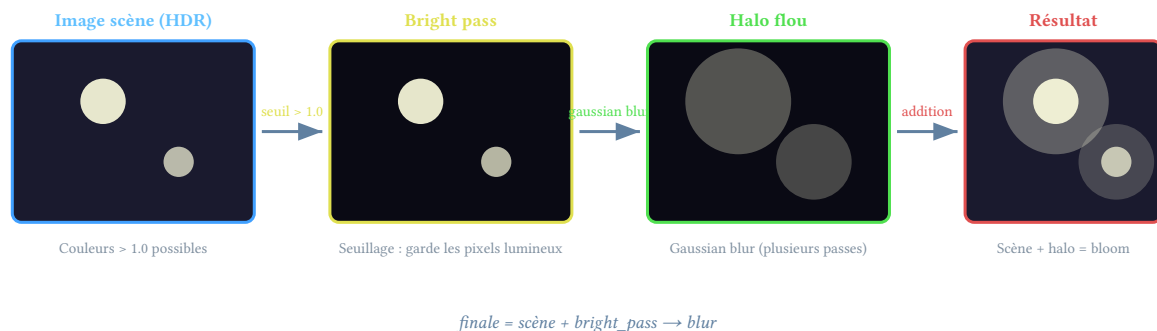


Figure 3: Le bloom en trois étapes : extraction des parties brillantes (bright pass), flou gaussien, puis addition à l'image originale.

❓ Pourquoi plusieurs passes de flou ?

Un gaussian blur sur un grand rayon coûte cher (beaucoup d'échantillons). On fait plusieurs passes avec un petit noyau (ex. 9×9) en alternant horizontal/vertical – c'est la **séparabilité** du gaussien. Pour un bloom large, on peut aussi **downsampler** l'image (la réduire en résolution), flouter, puis l'upsampler.

4. Depth of Field – le flou de profondeur

Principe

La caméra a une **focale** et un **ouverture**. En photographie, les objets hors de la zone nette (le **focal range**) sont flous. En temps réel, on reproduit cet effet en floutant les fragments dont la profondeur (le z du depth buffer) s'éloigne de la distance focale.

$$\text{flou} = \max(0, |z - \text{focalDistance}| - \text{focalRange})$$

💡 On réutilise le depth buffer

Le depth buffer (Session 3, Session 10) est disponible comme texture dans le post-processing. On lit z au pixel courant, on calcule la distance au focal point, et on ajuste le rayon de flou. C'est le même depth buffer que pour le shadow mapping – on l'a déjà, il est gratuit.

5. Screen-Space Ambient Occlusion (SSAO)

Principe

L'AO (Session 6) assombrit les crevasses et coins où la lumière ambiante peine à arriver. Le SSAO le calcule en temps réel, dans l'espace écran, en échantillonnant le depth buffer autour de chaque pixel. Si beaucoup de voisins sont plus proches (crevasse), le pixel est assombri.

💡 Connexion avec la Session 6

La Session 6 a vu l'AO **baked** (précalculé dans une texture, statique). Le SSAO est la version **temps réel** — dynamique, mais approximative et plus coûteuse. Beaucoup de jeux combinent les deux : AO baked pour la géométrie statique + SSAO pour les objets dynamiques.

6. Autres effets courants

Effets simples (un seul passage)

- **Vignette** : assombrir les coins de l'image.

$$\text{color} * = 1.0 - \text{vignette}(\text{uv})$$

- **Color grading / LUT** : appliquer une table de correspondance de couleurs (look cinématique, désert, nuit).
- **Chromatic aberration** : décaler légèrement R, V, B (effet de lentille bon marché).
- **Film grain** : ajouter du bruit aléatoire (effet vintage).
- **Fog** : mélanger avec une couleur de brouillard selon la profondeur.

Partie 3 : Implémentation en Three.js

1. EffectComposer — la chaîne de montage

Le pattern de Three.js

Three.js fournit EffectComposer qui gère le ping-pong rendering automatiquement. On ajoute des **passes** — chaque pass est un effet. Le composer enchaîne les passes et affiche le résultat final.

```
import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';
import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';

const composer = new EffectComposer(renderer);
composer.addPass(new RenderPass(scene, camera)); // passe 1 : scène
composer.addPass(new UnrealBloomPass(resolution, strength, radius, threshold));
// ... ajouter d'autres passes ...

// Dans la boucle de rendu :
composer.render(); // au lieu de renderer.render(scene, camera)
```

2. Les passes intégrées

Pass disponibles dans Three.js

- **RenderPass** — rend la scène dans le render target (toujours en premier).
- **UnrealBloomPass** — bloom de style Unreal Engine.
- **ShaderPass** — passe custom avec votre propre shader.
- **BokehPass** — depth of field.
- **SAOPass** — screen-space ambient occlusion.
- **OutputPass** — tone mapping + gamma (toujours en dernier).

3. ShaderPass — un effet custom

Vignette en ShaderPass.

```
import { ShaderPass } from 'three/addons/postprocessing/ShaderPass.js';

const vignettePass = new ShaderPass({
  uniforms: {
    tDiffuse: { value: null }, // image d'entrée (automatique)
    offset: { value: 1.0 },
    darkness: { value: 1.0 }
  },
  vertexShader: `
    varying vec2 vUV;
    void main() {
      vUV = uv;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragmentShader: `
    uniform sampler2D tDiffuse;
    uniform float offset;
    uniform float darkness;
    varying vec2 vUV;

    void main() {
      vec4 color = texture2D(tDiffuse, vUV);
      // Distance au centre
      vec2 uv = (vUV - 0.5) * offset;
      float vignette = 1.0 - dot(uv, uv) * darkness;
      gl_FragColor = color * vignette;
    }
  `
});

composer.addPass(vignettePass);
```

💡 Convention Three.js

tDiffuse est le nom conventionnel pour l'image d'entrée dans un ShaderPass. Le EffectComposer la remplit automatiquement avec le render target de la pass précédente. Ne pas la renommer.

4. Ordre des passes

💬 L'ordre compte

L'ordre des passes affecte le résultat :

1. RenderPass — toujours en premier.
2. Effets qui travaillent en HDR (bloom, DoF, SSAO) — **avant** le tone mapping.
3. OutputPass (tone mapping + gamma) — **toujours en dernier**.

Si on met le tone mapping avant le bloom, le bloom travaille en LDR et les halos seront plats. Si on le met après, le bloom peut exploiter la plage HDR complète.

Partie 4 : Performance et Bonnes Pratiques

Coût du post-processing

Chaque pass lit et écrit une texture plein écran. En 1080p, c'est $\sim 2M$ pixels par pass. Le coût principal est la **bande passante mémoire** (lire/écrire la texture), pas le calcul.

- **Réduire la résolution** : certains effets (bloom, SSAO) peuvent tourner à demi-résolution — visuellement identique, 4× moins de pixels.
- **Combiner les effets** : vignette + color grading + grain dans un seul shader au lieu de trois passes.
- **Éviter les passes inutiles** : si un effet est constant (ex. vignette fixe), on peut le baker dans une LUT.

💡 Le fullscreen quad est gratuit

Le vertex shader du fullscreen quad ne traite que 4 à 6 vertices — négligeable. C'est le fragment shader (un par pixel) qui coûte. Donc la complexité du fragment shader est ce qui compte, pas la géométrie.

Partie 5 : Démonstration en Direct — Three.js

💬 Démo interactive

On part de la démo de shadows de la Session 10 et on ajoute du post-processing :

1. Tone mapping ACES (via `OutputPass`).
2. Bloom sur les zones lumineuses (`UnrealBloomPass`).
3. Vignette custom (`ShaderPass`).
4. Un slider pour ajuster l'intensité du bloom en temps réel.

On voit l'effet de chaque pass sur l'image finale, et l'impact sur le framerate.

Partie 6 : Pratique — Ajouter du Post-Processing

Cahier des charges. Reprendre une scène Three.js (la vôtre ou celle de la Session 10) et y ajouter :

1. Tone mapping (ACES ou Reinhard).
2. Bloom avec un seuil réglable.
3. Au moins un effet custom (vignette, color grading, ou chromatic aberration).
4. Un contrôle GUI pour ajuster les paramètres en temps réel.

Étapes — Three.js

1. Installer `EffectComposer` et les passes nécessaires.
2. Remplacer `renderer.render()` par `composer.render()`.
3. Ajouter `RenderPass` en premier, `OutputPass` en dernier.
4. Insérer les effets entre les deux.
5. Pour un effet custom, créer un `ShaderPass` avec un shader GLSL.
6. Ajuster les paramètres via `lil-gui` ou `dat.gui`.

Critères de réussite

1. Le tone mapping est visible (les hautes lumières ne sont plus écrêtées).
2. Le bloom est visible sur les sources lumineuses.
3. L'effet custom est visible et paramétrable.

4. Le framerate reste acceptable (≥ 30 FPS).
5. L'ordre des passes est correct (tone mapping en dernier).

Conclusion

Ce qu'on a vu

- Le post-processing applique des effets sur l'image **après** le rendu de la scène.
- Le pipeline : rendre dans un render target → enchaîner les effets (ping-pong) → afficher.
- Les effets clés : tone mapping (HDR→LDR), bloom (halo lumineux), DoF (flou de profondeur), SSAO (occlusion ambiante).
- En Three.js : EffectComposer + passes (RenderPass, UnrealBloomPass, ShaderPass, OutputPass).
- L'ordre des passes compte : tone mapping toujours en dernier.

Le post-processing est la couche finale du rendu — il transforme une image techniquement correcte en une image **belle**. C'est ce qui donne le "look" d'un jeu.

La prochaine session

En Session 12, on regarde **sous le capot** : WebGPU, la nouvelle API graphique du web. On a utilisé Babylon.js et Three.js qui abstraient le GPU — maintenant on voit ce qu'il y a en dessous, et pourquoi WebGPU change la donne (compute shaders, performance, accès direct au GPU).